AI, ML, Java Digital Assignment

?) What is our project about?

We will implement a Reactive AI Agent that detects fraudulent transactions trained on creditcard transactions and send an email to a client email from an agent email based on if fraudulent transaction was detected or not

?) What are our components?

For AI, the overall project will be a Reactive AI Agent where:

- Environment is a stream of transactions that the AI Agent will be processing
- Inferences will be made using an XGBoost model trained to detect fraudeulent transactions
- Action will be taken in the form of mail sent from AI Agent Email to a Client Email when a fraudulent transaction is detected.

For Java, we will be using SpringBoot as middleware between frontend and backend. Kafka is also a Java Technology

For Ml, we will be training a model to detect fraudulent transaction based on anonymised credit card transaction dataset.

How to run it?

1. **Run Apache Zookeeper**
   Zookeeper is located at *"C:\kafka_2.12-3.9.0\bin\windows\zookeeper-server-start.bat"*.

   Open CMD. Go to "*C:\kafka_2.12-3.9.0"*

   Run the cmd *".\bin\windows\zookeeper-server-start.bat .\config\zookeeper.properties"*

   That should run zookeeper.

2. **Run Apache Kafka**
   Kafka is located at *"C:\kafka_2.12-3.9.0\bin\windows\kafka-server-start.bat"*.

   Open CMD. Go to *"C:\kafka_2.12-3.9.0"*

   Run the cmd *".\bin\windows\kafka-server-start.bat .\config\server.properties"*

   Now kafka will be running.

3. **Run Flask API which hosts the XGBoost Model**
   Go to *"D:\VIT Projects\AI, ML, Java Digital Assignment\Deploying Model as REST API (Flask)\app.py"*

   Open app.py.

   Run app.py

   That should now successfully host the XGBoost model at *http://localhost:5000*

4. **Stream the transaction data from test dataset to producer**
   Go to *"D:\VIT Projects\AI, ML, Java Digital Assignment\data streaming\streaming_data_from_csv_file _into_producer\stream_to_producer.py*"

   Open stream_to_producer.py

   Run stream_to_producer.py
   That should run the producer.

5. **Stream data from consumer and sent it to Springboot using Websocket**

Go to *"D:\VIT Projects\AI, ML, Java Digital Assignment\data streaming\streaming_data_into_model_from consumer\stream_to_consumer.py"*

Open stream_to_consumer.py

Run stream_to_consumer.py

That should run the consumer

6. Run Springboot

Open the folder *"D:\VIT Projects\AI, ML, Java Digital Assignment\fraud-detection"* in IntelliJ IDEA 2024.1 and run it

That should run the Springboot project

Go to *http://localhost:8081* to see the live fraud transaction streams

Code

D:\VIT Projects\AI, ML, Java Digital Assignment\Deploying Model as REST API (Flask)\app.py (app.py)

```
from flask import Flask, request, jsonify
import joblib
import numpy as np
import pandas as pd
from flask_cors import CORS

# Load the trained XGBoost model
model = joblib.load("Model/XGBoost/xgboost_fraud_model.pkl")

app = Flask(__name__)
CORS(app)

# Define the correct feature list
feature_list = ['V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9',
                'V10', 'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18',
                'V19', 'V20', 'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount']

@app.route('/predict', methods=['POST'])
def predict():
    try:
        data = request.json
        if not data:
            return jsonify({"error": "Empty request data"}), 400

        df = pd.DataFrame(data if isinstance(data, list) else [data])

        missing_features = [feature for feature in feature_list if feature not in
df.columns]
```

```
        if missing_features:
            return jsonify({"error": f"Missing features: {missing_features}"}), 400

        if 'id' in df.columns:
            df.pop('id')

        df = df[feature_list]
        features = df.to_numpy()

        predictions = model.predict(features)
        probabilities = model.predict_proba(features)[:, 1]

        response = [{"fraud": bool(pred), "probability": float(prob)}
                    for pred, prob in zip(predictions, probabilities)]

        return jsonify(response)

    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5000)
```

D:\VIT Projects\AI, ML, Java Digital Assignment\data streaming\streaming_data_from_csv_file_into_producer\stream_to_producer.py (stream_to_producer.py)

```
from kafka import KafkaProducer
import pandas as pd
import json
import time

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

df = pd.read_csv("content/creditcard_test.csv")

for _, row in df.iterrows():
    message = row.to_dict()
    producer.send("fraud_transactions", value=message)
    print(f"Sent: {message}")
    time.sleep(1)  # Simulating real-time streaming

producer.close()
```

D:\VIT Projects\AI, ML, Java Digital Assignment\data streaming\streaming_data_into_model_from consumer\stream_to_consumer.py(stream_to_consumer.py)

```
import requests
import json
import time
import smtplib
from kafka import KafkaConsumer
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

# ✅ Fraud Prediction API & Spring Boot WebSocket API
API_URL = "http://127.0.0.1:5000/predict"
SPRING_BOOT_API = "http://localhost:8081/api/update"

# ✅ Email Config (SMTP)
SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587
SMTP_USERNAME = "fraud.detection.agent.2024@gmail.com"
SMTP_PASSWORD = "bbweqeiqmbgsynsh"  # Replace with your Gmail app password
EMAIL_SENDER = SMTP_USERNAME
EMAIL_RECEIVER = "fraud.detection.client.2024@gmail.com"

# ✅ Fraud email counter
fraud_email_count = 0
FRAUD_EMAIL_LIMIT = 3

def send_alert(transaction, probability):
    """Send fraud alert email using SMTP"""
```

```
    subject = "🚨 Fraud Alert: Suspicious Transaction Detected!"
    body = f"""
    <html>
    <body>
        <h2>🚨 Fraud Alert Detected!</h2>
        <p><b>Transaction Details:</b> {transaction}</p>
        <p><b>Fraud Probability:</b> {probability:.2f}</p>
        <p><b>Action:</b> Blocked ✅</p>
    </body>
    </html>
    """

    msg = MIMEMultipart("alternative")
    msg["Subject"] = subject
    msg["From"] = f"Fraud Detection <{EMAIL_SENDER}>"
    msg["To"] = EMAIL_RECEIVER
    msg.attach(MIMEText(body, "html"))

    try:
        server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
        server.starttls()
        server.login(SMTP_USERNAME, SMTP_PASSWORD)
        server.sendmail(EMAIL_SENDER, EMAIL_RECEIVER, msg.as_string())
        server.quit()
        print("✅ Alert sent via SMTP.")
    except Exception as e:
        print(f"❌ Email alert failed: {e}")

# ✅ Kafka Consumer Configuration
consumer = KafkaConsumer(
```

```
    'fraud_transactions',
    bootstrap_servers='localhost:9092',
    auto_offset_reset='earliest',
    value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)

# ✅ Process Kafka messages
for message in consumer:
    transaction_data = message.value
    print(f"Received: {transaction_data}")

    response = requests.post(API_URL, json=transaction_data)

    if response.status_code == 200:
        prediction = response.json()
        print(f"Prediction: {prediction}")

        # ✅ If fraudulent and within email limit
        if prediction[0]['fraud']:
            if fraud_email_count < FRAUD_EMAIL_LIMIT:
                send_alert(transaction_data, prediction[0]['probability'])
                fraud_email_count += 1
            else:
                print("🚫 Fraud email limit reached. No more alerts will be sent.")

        # ✅ Send to WebSocket
        requests.post(SPRING_BOOT_API, json={"transaction": transaction_data,
"prediction": prediction})
        print("✅ Sent data to Spring Boot WebSocket")
```

```
else:
    print(f"❌ Error: {response.text}")

time.sleep(1)
```

D:\VIT Projects\AI, ML, Java Digital Assignment\fraud-detection\app\src\main\resources\static\index.html (index.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>Fraud Detection</title>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/sockjs-client/1.5.1/sockjs.min.js"></script>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/stomp.js/2.3.3/stomp.min.js"></script>

  <script>
    document.addEventListener("DOMContentLoaded", function () {
      const socket = new SockJS('/fraud-websocket');
      const stompClient = Stomp.over(socket);

      stompClient.connect({}, function () {
        console.log("✅ Connected to STOMP WebSocket!");

        stompClient.subscribe('/topic/transactions', function (message) {
          const data = JSON.parse(message.body);
          const transactionDiv = document.createElement("div");
          transactionDiv.innerHTML = `<b>Transaction:</b> ${JSON.stringify(data.transaction)}
                                      <br><b>Prediction:</b> ${data.prediction[0].fraud ? '<span style="color:red">🚨 Fraud</span>' : '<span style="color:green">✅ Safe</span>'}
                                      <br><b>Probability:</b> ${data.prediction[0].probability.toFixed(4)}`;

          document.getElementById("transactions").prepend(transactionDiv);

          if (data.prediction[0].fraud) {
            alert("⚠️ Fraud detected! Taking action.");
          }
        });
      });
    });
  </script>
</head>
<body>
<h3>📡 Real-Time Transactions:</h3>
<section id="transactions"></section>
</body>
</html>
```

D:\VIT Projects\AI, ML, Java Digital Assignment\fraud-detection\app\src\main\resources\application.properties(application.properties)

```
# MySQL Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/fraud_detection?useSSL=false&
allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver


# Hibernate & Connection Pooling
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.datasource.hikari.minimum-idle=2
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.idle-timeout=30000
spring.datasource.hikari.connection-timeout=20000
spring.datasource.hikari.max-lifetime=1800000

# Hibernate & JPA Settings
spring.jpa.open-in-view=false

# Server Configuration
server.port=8081

# Thymeleaf Configuration
spring.thymeleaf.cache=false
spring.thymeleaf.prefix=classpath:/templates/
spring.thymeleaf.suffix=.html

# Logging (Enable Debugging)
logging.level.org.springframework=DEBUG

# Enable Actuator Endpoints
management.endpoints.web.exposure.include=*
management.endpoint.web.exposure.include=*

logging.level.org.springframework.web=DEBUG
logging.level.org.springframework.messaging=DEBUG
```

D:\VIT Projects\AI, ML, Java Digital Assignment\fraud-detection\app\build.gradle(build.gradle)

```
plugins {
    id 'java'
    id 'application'
    id 'org.springframework.boot' version '3.2.1'  // ✅ Required for bootRun
    id 'io.spring.dependency-management' version '1.1.4'
}

group = 'com.fraud.detection'
version = '1.0.0'

repositories {
    mavenCentral()
}

dependencies {
    implementation project(':ml-library') // ✅ Include ML library

    // ✅ Spring Boot dependencies
    implementation 'org.springframework.boot:spring-boot-starter-websocket'  // ✅ Required for WebSockets
    implementation 'org.springframework.boot:spring-boot-starter-web'       // ✅ Ensure Web support is enabled
    implementation 'org.springframework.boot:spring-boot-starter'
    implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
    implementation 'org.springframework.boot:spring-boot-starter-json'
    implementation 'com.fasterxml.jackson.core:jackson-databind:2.15.3'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-actuator'
    implementation 'mysql:mysql-connector-java:8.0.33'
    implementation 'org.springframework.kafka:spring-kafka'

    // ✅ Machine Learning dependencies
    implementation 'nz.ac.waikato.cms.weka:weka-stable:3.8.6'

    // ✅ Lombok for reducing boilerplate
    compileOnly 'org.projectlombok:lombok'
    annotationProcessor 'org.projectlombok:lombok'

    // ✅ Testing dependencies
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}
```

```
tasks.named('test') {
    useJUnitPlatform()
}

java {
    toolchain {
        languageVersion.set(JavaLanguageVersion.of(17))
    }
    sourceCompatibility = '17'  // Ensure Java 17 if using JDK 17
}

bootJar {
    mainClass = 'com.fraud.detection.app.FraudDetectionApplication' // ✅ Ensure this is correct
}

application {
    mainClass = 'com.fraud.detection.app.FraudDetectionApplication' // ✅ Ensure this is correct
}
```

D:\VIT Projects\AI, ML, Java Digital Assignment\fraud-detection\app\src\main\java\com\fraud\detection\app\FraudDetectionApplication.java (FraudDetectionApplication.java)

```
package com.fraud.detection.app;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.web.client.RestTemplate;

@SpringBootApplication(scanBasePackages = "com.fraud.detection") // Ensure this is correct
public class    FraudDetectionApplication {
   public static void main(String[] args) {
      SpringApplication.run(FraudDetectionApplication.class, args);
   }

   @Bean
   public RestTemplate restTemplate() {
      return new RestTemplate();
   }
}
```

D:\VIT Projects\AI, ML, Java Digital Assignment\fraud-detection\app\src\main\java\com\fraud\detection\config\WebSocketConfig.java (WebSocketConfig.java)

```
package com.fraud.detection.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import
org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroke
r;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import
org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfig
urer;

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/topic");
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/fraud-websocket").withSockJS();
    }
}
```

D:\VIT Projects\AI, ML, Java Digital Assignment\fraud-detection\app\src\main\java\com\fraud\detection\controllers\FraudDetectionController.java (FraudDetectionController.java)

```
package com.fraud.detection.controllers;

import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.web.bind.annotation.*;
import java.util.Map;

@RestController
@RequestMapping("/api")
public class FraudDetectionController {
   private final SimpMessagingTemplate messagingTemplate;

   public FraudDetectionController(SimpMessagingTemplate messagingTemplate) {
      this.messagingTemplate = messagingTemplate;
   }

   @PostMapping("/update")
   public void sendUpdateToFrontend(@RequestBody Map<String, Object>
predictionData) {
      messagingTemplate.convertAndSend("/topic/transactions", predictionData);
   }
}
```

D:\VIT Projects\AI, ML, Java Digital Assignment\fraud-detection\app\src\main\java\com\fraud\detection\services\FraudDetectionService.java (FraudDetectionService.java)

```
package com.fraud.detection.services;

import org.springframework.http.*;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import com.fasterxml.jackson.databind.ObjectMapper;

import java.util.Map;
import java.util.logging.Logger;

@Service  // Ensure this annotation is present
public class FraudDetectionService {
   private static final String ML_API_URL = "http://localhost:5000/predict";
   private static final Logger logger =
Logger.getLogger(FraudDetectionService.class.getName());

   private final RestTemplate restTemplate;

   public FraudDetectionService(RestTemplate restTemplate) {
      this.restTemplate = restTemplate;
   }

   public String checkFraud(Map<String, Object> transactionData) throws Exception
{
      HttpHeaders headers = new HttpHeaders();
      headers.setContentType(MediaType.APPLICATION_JSON);

      String jsonRequest = new
ObjectMapper().writeValueAsString(transactionData);
      HttpEntity<String> entity = new HttpEntity<>(jsonRequest, headers);

      ResponseEntity<String> response =
restTemplate.postForEntity(ML_API_URL, entity, String.class);
      return response.getBody();
   }
}
```

LocalHosts

1. Flask API (ML Fraud Detection)

- **Runs on:** http://0.0.0.0:5000 (accessible via http://localhost:5000)
- **Port:** 5000
- **Purpose:**
  - Hosts the XGBoost fraud detection model.
  - Receives JSON transaction data via a **POST request to /predict**.
  - Returns fraud detection predictions.

- **Access Flask API:**

Test the API in **Postman** or with **cURL**:

curl -X POST "http://localhost:5000/predict" -H "Content-Type: application/json" -d '{

"V1": 1.2, "V2": -0.5, ..., "V28": -0.2, "Amount": 50

}'

2. Kafka Broker

- **Runs on:** http://localhost:9092
- **Port:** 9092
- **Purpose:**
  - Kafka serves as a message queue.
  - The **Kafka Producer (stream_to_producer.py)** streams transactions to the **Kafka topic** (fraud_transactions).
  - The **Kafka Consumer (stream_to_consumer.py)** listens to the fraud_transactions topic and forwards data to the Flask API.

- 💡 **Check if Kafka is running:**
  netstat -an | findstr 9092

3. Spring Boot WebSocket API

- **Runs on:** http://localhost:8081
- **Port:** 8081
- **Purpose:**
  - The **Spring Boot WebSocket (/fraud-websocket)** is used for **real-time communication**.
  - The Kafka consumer **sends results to** the **Spring Boot API (http://localhost:8081/api/update)**.
  - The front-end **subscribes to /topic/transactions** to receive live fraud predictions.
- 💡 **Check if Spring Boot is running:**
  Open http://localhost:8081/test in a browser.

4. Front-End

- **Runs on:** http://localhost:8081
- **Port:** 8081
- **Purpose:**
  - Uses **SockJS & STOMP WebSocket** to subscribe to **real-time fraud transactions**.
  - Connects to the WebSocket at http://localhost:8081/fraud-websocket.

💡 **How it works:**

1. **User loads the webpage.**
2. **WebSocket connects** (SockJS('/fraud-websocket')).
3. **It subscribes to /topic/transactions.**
4. **Spring Boot pushes live transaction results**.

Pipeflow

Step 1: Data from creditcard_test.csv that contains transaction data is fed into Kafka Producer. The data is supposed to simulate transactions that happens in real time.

Step 2: Kafka Producer(*stream_to_producer.py*) sends the transactions row by row to a Kafka topic broker *fraud_transactions* every second. This simulates a real time stream of transactions.

Step 3: Kafka Consumer(*stream_to_consumer.py*) is subscribed to *fraud_transactions*. This makes it so that the transactions received by the broker is extracted by Kafka Consumer.

Step 4: An XGBoost Model is trained on creditcard dataset to predict fraudulent transactions.

Step 5: The transactions extracted by Kafka Consumer is sent to the trained model using Flask API(*app.py*) which returns a probability and whether the transaction was fraudulent or not.

Step 6: If the transaction was fraudulent, then an email is sent from an AI Agent Email(*fraud.detection.agent.2024@gmail.com*) to a client email(*fraud.detection.client.2024@gmail.com*) using SMTP protocol.

Step 7: The transaction data received by Kafka Consumer and the prediction is also sent to a SpringBoot REST

API(*FraudDetectionController.java*). Using *SimpMessagingTemplate*, the SpringBoot Controller broadcasts the transactions & predictions to the Web UI using WebSocket(*/topic/transaction*)

Step 8: The HTML connects to */fraud-websocket*. It listens to */topic/transactions*. When the message is received, it renders the Transaction details, Prediction and Probability.

Technologies Used

Layer	Technology Used
Data Source	CSV(offline simulation of transactions)
Streaming	Apache Kafka(Producer & Consumer)
ML API	Flask + joblib + XGBoost
Agent Reaction	Python + SMTP(Email)
Backend Middleware	Java(SpringBoot + WebScoket)
Frontend	HTML + STOMP + SockJS

Components for each subject

Subject	Component
Java	SpringBoot frontend + backend & Kafka
ML	XGBoost training and testing
AI	The whole project works by detecting fraudulent transactions by processing incoming transactions and predicting whether they are fraudulent or not. This makes it a Reactive Agent that works in an environment that processes credit card transactions, makes inferences based on the prediction and takes an action based on the inference

Architecture Diagram